

OSDBMS-V-2026-T038

SYSTEM ARCHITECTURE & DESIGN DOCUMENT

WALnut

A Concurrent Auction Engine with Deadlock Detection & Recovery

Team: The Acid Mechanics – T038

Vinayak Suyal · Sushmit Singh Rawat · Kanak Rawat · Suhavi Jugran

Version 1.0 · Companion to the WALnut PRD

Status: Draft for Review

Table of Contents

- 1. Introduction
- 2. Architectural Goals & Constraints
- 3. High-Level Architecture
- 4. Component Descriptions
- 5. Data Model & Schema Design
- 6. Storage Layer Design
- 7. Buffer Pool Design
- 8. Concurrency Control Design
- 9. Deadlock Detection & Recovery Design
- 10. Write-Ahead Logging & Crash Recovery
- 11. API & Communication Layer
- 12. Technology Stack
- 13. Key Design Decisions & Trade-offs
- 14. Future Extensions (Stretch Goals)

1. Introduction

1.1 Purpose

This document specifies how WALnut is built: its components, their responsibilities, the interfaces between them, and the internal designs of the storage, concurrency, deadlock-handling, and recovery subsystems. It is the companion to the WALnut PRD, which defines what the system must do; this document defines how.

1.2 Audience

The four members of The Acid Mechanics (T038), each owning a primary subsystem, and course evaluators assessing design quality against OS and DBMS learning outcomes.

1.3 Relationship to Other Documents

Functional and non-functional requirements are defined in the PRD and are not restated in full here. This document elaborates Section 8 (Data Model) and Section 9 (System Overview) of the PRD into concrete, buildable designs.

2. Architectural Goals & Constraints

2.1 Goals

- Observability first: every internal mechanism (locks, wait-for graph, buffer pool, WAL) must be inspectable live, not just correct in the background.
- Correctness under concurrency: Strict 2PL must make every placeBid transaction serializable with respect to every other transaction on the same item.
- Deterministic demonstrability: the deadlock and recovery mechanisms must be triggerable on demand via fixed test scenarios (reserveBundle, fault injection), not left to chance timing.
- Subsystem independence: storage, buffer pool, lock manager, deadlock detector, and recovery manager should be separately unit-testable behind clear interfaces.

2.2 Constraints

- C++17, built from scratch — no third-party embedded database (e.g. SQLite, LevelDB) may be used for the engine core.
- Single-process, single-machine deployment; no distributed consensus or replication.
- Eight-week build window aligned to the milestone plan in the PRD.
- No authentication layer; identity is a fixed user_id chosen by the operator or test script.

3. High-Level Architecture

WALnut is organized as a layered stack: a client dashboard talks to an embedded REST API, which invokes a fixed set of auction operations, which run as transactions coordinated by a transaction/lock manager over a buffer pool and storage manager, with a deadlock manager and recovery manager cutting across the transaction layer.

WALnut — System Architecture (High Level)

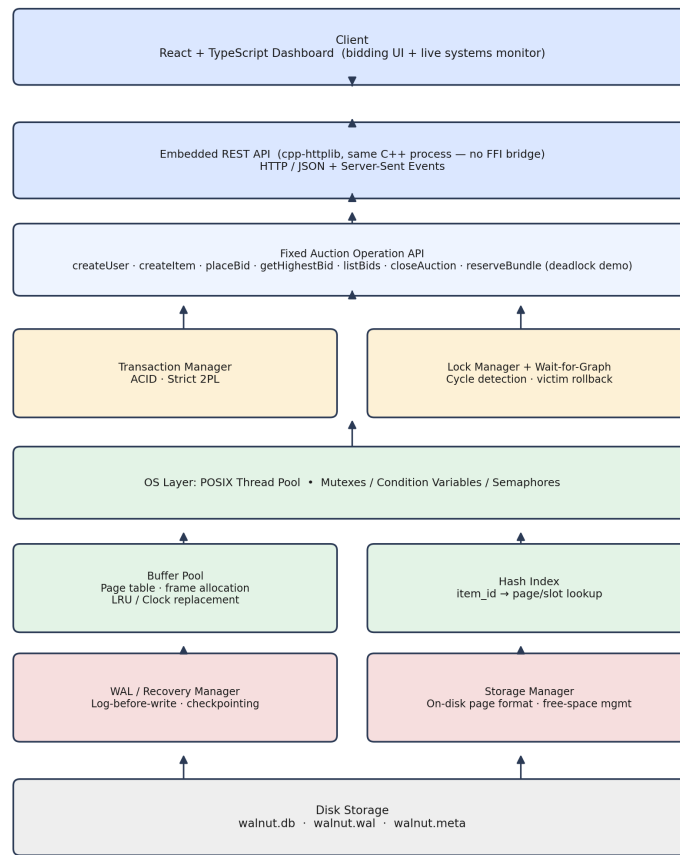


Figure 1. WALnut high-level layered architecture.

4. Component Descriptions

Component	Responsibility
Client Dashboard (React + TypeScript)	Bidding view (place bids, view highest bid / history) and systems-monitor view (lock table, wait-for graph, buffer stats, WAL activity). Consumes the REST API and an SSE stream for live updates.
Embedded REST API (cpp-http lib)	Runs in the same process as the engine — no FFI/IPC boundary. Exposes the fixed operation API over HTTP/JSON and pushes live telemetry over Server-Sent Events.
Fixed Operation API	createUser, createItem, placeBid, getHighestBid, listBids, closeAuction, reserveBundle. Each call is compiled internally to simple relational-algebra steps (SELECT / PROJECT / UPDATE) against the schema.
Transaction Manager	Begins/commits/aborts transactions; sequences lock acquisition, the operation's reads/writes, WAL record emission, and lock release for every fixed operation.
Lock Manager	Grants/queues shared and exclusive locks per item under Strict Two-Phase Locking; maintains the lock table that the dashboard reads for its live view.

Component	Responsibility
Deadlock Manager (Wait-for Graph)	Maintains the wait-for graph from the lock manager's queue state; periodically scans for cycles; on detection, selects and aborts a victim, releases its locks, and triggers a retry.
Buffer Manager (Buffer Pool)	Caches fixed-size pages in memory via a page table; pin/unpin semantics; Clock or LRU eviction of unpinned pages; tracks dirty pages for write-back and hit/miss statistics.
Index Manager (Hash Index)	Maintains a hash index on item_id for O(1) average-case lookup of an item's current state, avoiding full page scans.
Recovery Manager (WAL)	Writes before/after-image log records ahead of every page flush; on restart, redoes committed transactions and undoes uncommitted ones from the log; owns checkpointing.
Storage Manager	Defines the on-disk fixed-size page format, allocation and free-space management, and raw file I/O against walnut.db, walnut.wal, and walnut.meta.
OS / Concurrency Layer	POSIX thread pool that executes incoming operations; mutexes, condition variables, and semaphores coordinate the lock manager's wait/notify behavior and buffer pool access.

5. Data Model & Schema Design

5.1 Entity-Relationship Overview

Three relations, normalized to Third Normal Form. An Item owns its own auction state (current_bid, current_winner, status) rather than delegating it to a separate Auctions relation — sufficient to demonstrate keys, functional dependencies, and normalization without unnecessary schema surface.

Relation	Attribute	Type / Notes
Users	user_id (PK)	Fixed identifier, no auth
	username	Display name
Items	item_id (PK)	Fixed identifier
	title	Item description
	starting_price	Initial asking price
	status	OPEN / CLOSED
	current_bid	Denormalized highest bid amount
	current_winner	FK → Users.user_id (denormalized)
Bids	bid_id (PK)	Auto-increment
	item_id	FK → Items.item_id
	user_id	FK → Users.user_id
	amount	Bid amount

Relation	Attribute	Type / Notes
	bid_time	Timestamp

5.2 Functional Dependencies

- Users: user_id → username
- Items: item_id → title, starting_price, status, current_bid, current_winner
- Bids: bid_id → item_id, user_id, amount, bid_time

current_bid and current_winner in Items are functionally dependent on item_id but are also derivable from MAX(amount) over Bids for that item_id. This is a deliberate, documented denormalization: it turns getHighestBid into an O(1) indexed lookup instead of an aggregate scan over Bids, at the cost of requiring placeBid to keep the two in sync within the same transaction — which Strict 2PL already guarantees is atomic.

6. Storage Layer Design

6.1 Page Format

The storage manager operates on fixed-size pages (e.g. 4 KB). Each page has a header (page ID, page type, slot count, free-space pointer) followed by a slotted-page body holding variable-length records (Bids) or fixed-length records (Users, Items), consistent with the record layout used across the Items/Bids/Users tables.

6.2 File Layout

File	Contents
walnut.db	Fixed-size data pages: Users, Items, Bids tables and the hash index.
walnut.wal	Append-only Write-Ahead Log of before/after-image records and commit/abort markers.
walnut.meta	Metadata: page allocation bitmap, next available page ID, checkpoint pointers.

6.3 Free-Space & Allocation

A free-page list (or allocation bitmap in walnut.meta) tracks which pages are in use. New records that don't fit in an existing page's free space trigger allocation of a new page from this list.

7. Buffer Pool Design

The buffer pool is a fixed number of in-memory frames, each holding one page, indexed by a page table (page ID → frame number).

- pin(page_id) increments a frame's pin count and loads the page from disk if not already cached.
- unpin(page_id, dirty) decrements the pin count and marks the frame dirty if the page was modified.
- A frame with pin count 0 is eligible for eviction; frames with pin count > 0 are never evicted.
- Replacement policy: Clock (second-chance) or LRU — one is selected as the implemented policy per the PRD's core scope; the other remains a stretch comparison.

- A dirty evicted page is flushed to disk only after its corresponding WAL record has been written (write-ahead rule), never before.
- Hit/miss counters are exposed to the dashboard for the buffer-pool statistics view; low frame counts are used deliberately in testing to make thrashing observable.

8. Concurrency Control Design

8.1 Strict Two-Phase Locking

Every placeBid transaction acquires an exclusive lock on its target item before reading or writing any of that item's state, and holds all locks until commit or abort (the “strict” variant — no early release). This guarantees no transaction observes another's uncommitted changes and prevents cascading aborts.

8.2 Lock Manager Internals

- Lock table: $\text{item_id} \rightarrow \{ \text{holder(s)}, \text{lock mode}, \text{wait queue of blocked transaction IDs} \}$.
- `acquireLock(txn_id, item_id, mode)` grants immediately if compatible, otherwise enqueues the requester and blocks its thread on a condition variable.
- `releaseLock(txn_id, item_id)` on commit/abort wakes the next compatible waiter(s) via the condition variable.
- Every enqueue/dequeue event updates an edge in the wait-for graph maintained by the Deadlock Manager (Section 9).

8.3 Transaction Lifecycle (placeBid)

- 1. BEGIN — transaction manager allocates a transaction ID.
- 2. LOCK — acquire exclusive lock on `item_id` (may block; may enqueue a wait-for edge).
- 3. READ — fetch `current_bid` / `current_winner` via the buffer pool.
- 4. VALIDATE — reject if $\text{amount} \leq \text{current_bid}$.
- 5. WRITE — append Bid record; update `Items.current_bid` / `current_winner`.
- 6. LOG — write WAL record (before-image, after-image) before any dirty page is flushed.
- 7. COMMIT — mark transaction committed in the log; release all held locks.

9. Deadlock Detection & Recovery Design

9.1 Wait-for Graph

Nodes are active transaction IDs. An edge $T_i \rightarrow T_j$ exists whenever T_i is blocked waiting for a lock held by T_j . The graph is derived directly from the lock manager's wait queues, so it always reflects live lock state.

9.2 Detection Algorithm

- A background thread periodically (fixed interval) runs cycle detection (DFS-based) over the wait-for graph.

- On finding a cycle, a victim transaction is chosen (e.g. youngest transaction, or the one holding the fewest locks — policy documented in code comments and this section once finalized).
- The victim is aborted: its partial writes are undone via the WAL before-images, its locks are released, and its waiting edges are removed from the graph.
- The aborted transaction is automatically retried from BEGIN.

9.3 Deterministic Demonstration — reserveBundle

reserveBundle([item_A, item_B], user_id) is a deliberately constructed multi-lock operation. Running two overlapping reserveBundle calls that request the same two items in opposite order (T1: lock A, request B; T2: lock B, request A) produces a genuine circular wait, which the detector must catch and resolve within one detection interval. This is the primary acceptance test for this subsystem (see PRD Section 13, Week 6 gate).

9.4 Why Wait-for Graph over Banker's Algorithm

Banker's Algorithm requires each process to declare its maximum resource claim in advance so the system can verify a safe state before granting a request. A bidding transaction does not know every row it will touch before it runs, so this precondition cannot be honestly satisfied. Wait-for-graph detection is the standard mechanism for lock-based concurrency control of this kind, and is what PostgreSQL and MySQL/InnoDB use for the same reason.

10. Write-Ahead Logging & Crash Recovery

10.1 Log Record Format

Field	Purpose
LSN	Log sequence number, monotonically increasing
txn_id	Transaction that produced this record
type	UPDATE / COMMIT / ABORT / CHECKPOINT
page_id	Page affected (UPDATE records only)
before_image	Value prior to the update (for undo)
after_image	Value after the update (for redo)

10.2 Write-Ahead Rule

A page's UPDATE log record is durably written to walnut.wal before the corresponding dirty page may be flushed from the buffer pool to walnut.db. This is what makes recovery possible after a crash at any point.

10.3 Restart Recovery Algorithm

- 1. Scan the log from the last checkpoint forward, building the set of transactions that reached COMMIT.
- 2. Redo pass: reapply after-images for all UPDATE records belonging to committed transactions, in log order.

- 3. Undo pass: reapply before-images for all UPDATE records belonging to transactions that never committed (or were aborted), in reverse log order.
- 4. Write a new checkpoint once the database is consistent.

10.4 Fault-Injection Testing

Crash recovery is validated by deliberately killing the engine process at controlled points — mid-transaction and immediately after a page flush but before a WAL write, and vice versa — then verifying on restart that committed bids are intact and in-flight bids are fully undone (PRD Section 13, Week 7 gate).

11. API & Communication Layer

11.1 REST Endpoints (indicative)

Method & Path	Maps to	Notes
POST /users	createUser	
POST /items	createItem	
POST /items/{item_id}/bids	placeBid	Used by both the manual UI “Place Bid” control and load-test scripts — same path, no shortcut (PRD FR-3/FR-11).
GET /items/{item_id}/highest-bid	getHighestBid	
GET /items/{item_id}/bids	listBids	
POST /items/{item_id}/close	closeAuction	
POST /reserve-bundle	reserveBundle	Test/demo scenario endpoint for deadlock formation.
GET /stream (SSE)	—	Pushes lock-table, wait-for-graph, WAL, and buffer-pool events to the dashboard.

11.2 Live Telemetry Events (SSE)

- lock.acquired / lock.released / lock.waiting — drives the live lock table view.
- deadlock.detected / deadlock.resolved — drives the wait-for graph visualization.
- wal.write / checkpoint.complete — drives the WAL activity view.
- buffer.hit / buffer.miss — drives the buffer-pool statistics view.

12. Technology Stack

Layer	Technology	Purpose
Core engine	C++17	Storage manager, buffer pool, lock manager, transaction manager, WAL/recovery — all built from scratch.

Layer	Technology	Purpose
Concurrency	POSIX pthreads (mutexes, condition variables, semaphores)	Thread pool for concurrent bid handling and the lock manager's wait/notify logic.
API layer	Embedded REST server (cpp-httpplib), same process as the engine	Exposes the fixed operation API over HTTP/JSON — no cross-language bridge or shared-memory ownership issues.
Live updates	Server-Sent Events (SSE)	Pushes lock, transaction, deadlock, and WAL events to the dashboard in real time.
Persistence	Custom binary files via raw file I/O	walnut.db (fixed-size pages), walnut.wal (append-only log), walnut.meta (metadata).
Frontend	React + TypeScript + Vite	Type-safe bidding UI and systems-monitor dashboard.
Build & test	CMake, GoogleTest / Catch2	Reproducible builds and unit/concurrency tests for the engine.
Version control	Git + GitHub	Branching, review, and project tracking across the team.

13. Key Design Decisions & Trade-offs

Decision	Rationale	Trade-off Accepted
Fixed operation API instead of SQL	Removes the need for a parser/optimizer, keeping the 8-week scope focused on concurrency, storage, and recovery.	Not a general-purpose DBMS; limited query flexibility.
Strict 2PL over optimistic concurrency control	Simpler to reason about and demonstrate correctness for a small, high-contention workload (many bidders, one item).	More blocking under contention than OCC/MVCC.
Wait-for graph over Banker's Algorithm	Bidding transactions can't declare max resource claims in advance, so Banker's precondition can't be honestly met.	Requires a background detector thread and victim-selection policy.
Hash index (core) vs. B+ Tree (stretch)	O(1) point lookups are all getHighestBid needs; a B+ Tree adds range-query capability the fixed API doesn't use.	No efficient range scans over items; acceptable since none are required.
current_bid/current_winner denormalized into Items	Turns the hottest read path (getHighestBid) into an indexed lookup instead of an aggregate over Bids.	Requires placeBid to keep Items and Bids in sync — guaranteed by the same transaction's atomicity.
Embedded REST server in the same process	No FFI/IPC boundary, no struct marshaling or shared-memory ownership issues between engine and API.	API and engine share fate — an API crash is an engine crash (acceptable for a single-process demo).

14. Future Extensions (Stretch Goals)

Pursued only after the core requirements in the PRD are complete and demonstrated end-to-end:

- Replace the hash index with a B+ Tree to support range queries.
- Simulate SCAN / C-SCAN disk-request scheduling in the storage manager.
- Extend checkpointing beyond the basic recovery path (e.g. fuzzy checkpoints).
- Generalize reserveBundle into a full multi-item bundle transaction feature.
- Layer a tiny fixed SQL subset (single-table SELECT/INSERT/UPDATE) on top of the operation API.
- Deploy the engine and dashboard to a small persistent-disk VM for portfolio purposes.